QALA

Solution Factory Operating System

Software Design Document v1.0

Document Type	Software Design Document (SDD)
System	Qala Solution Factory OS
Version	1.0
Status	Draft
Audience	Engineering, Architecture, Product
Classification	Confidential

1 Executive Summary

Qala is an end-to-end Solution Factory Operating System for creating, managing, and evolving software solutions at scale.

Qala provides a structured operating environment that spans the full solution lifecycle — from initial conception through active production — within a networked fabric of Solution Development Environments (SDEs). Each SDE is a self-contained, configurable, and deployable workspace that defines all conditions required to create and maintain a solution. A collection of interconnected SDEs forms a Solution Factory (SF), enabling consistent, repeatable, and scalable solution production across teams and organisations.

At its core, Qala enforces a rigorous hierarchical solution structure: Systems contain Applications; Applications contain Processes; Processes contain Components; Components expose typed Interfaces; Interfaces carry Messages of well-defined Data Structures. This model ensures every artefact produced inside the platform is traceable, composable, and governable.

> *Qala is solution-agnostic. The platform does not prescribe technology stacks, programming languages, or delivery methodologies. Instead it defines the operational, structural, and governance conditions under which any solution type can be produced reliably.*

2 System Overview

2.1 Purpose

Qala serves as the operating system layer for solution factories. It provides:

- A canonical model for structuring solutions of any type.
- A managed environment (SDE) in which solutions are developed, configured, and operated.
- Orchestration and networking of multiple SDEs into a coordinated Solution Factory.
- Lifecycle governance including creation, versioning, deployment, maintenance, and decommissioning.
- Integrated intelligence (AI recommendations), security (SEM), and observability.

2.2 Solution Types

Qala supports six first-class solution types. Any deliverable produced inside the platform must be classified as one of the following:

Solution Type	Description
Application	A software application delivered to end-users or integrated systems.
System	A coordinated collection of applications serving a unified purpose.
Good	A tangible, physical, or digital deliverable produced by the factory.
Product	A commercially packaged, versioned, and distributable artefact.
Service	An ongoing capability delivered to consumers on a continuous basis.
Platform	A foundation on which other solutions are built and operated.

2.3 Design Principles

1. Composability: Every element of the solution model is composable and nestable.
2. Separation of concerns: Structure (solution model), environment (SDE), and orchestration (SF) are independent layers.
3. Solution-agnosticism: The OS imposes no language, framework, or methodology constraints.
4. Lifecycle completeness: The platform governs the full solution lifecycle without gaps.
5. Observability by default: All events, state changes, and operational signals are captured and streamed.
6. Security-first: Every SDE and solution is subject to continuous security monitoring and policy enforcement.

3 Solution Structure Model

Qala enforces a strict seven-level hierarchy that governs how every solution is decomposed and described.

3.1 Hierarchy

The solution structure model defines the following containment relationships:

Level	Element
1	System
2	Application (contained by System)
3	Process (contained by Application)
4	Component (contained by Process)
5	Interface (contained by Component)
6	Message (contained by Interface)
7	Data Structure (contained by Message)

3.2 Element Definitions

System

A System is the top-level organisational unit. It groups one or more Applications that together fulfil a coherent purpose. Systems provide the outermost governance boundary for lifecycle and access control.

Application

An Application is a deployable unit of functionality residing within a System. Applications encapsulate all Processes required to deliver their behaviour. Applications are independently versioned and can be replicated across SDEs.

Process

A Process defines a discrete unit of work executed within an Application. Processes are composed of one or more Components and represent the executable, observable units within the solution graph.

Component

A Component is a bounded, reusable building block within a Process. Components expose their capabilities and dependencies exclusively through typed Interfaces, ensuring clean boundaries and replaceability.

Interface

An Interface is the contract surface of a Component. Each Interface declares its Imports (inbound dependencies) and Exports (outbound capabilities). Interfaces are the primary integration point between Components and between external systems.

Interface Direction	Meaning
Import	Messages consumed by the Component through this Interface.
Export	Messages produced by the Component through this Interface.

Message

A Message is the unit of communication flowing through an Interface. Every Message has a type that determines its temporal behaviour:

Message Type	Behaviour
Event (dynamic)	Represents a discrete occurrence at a point in time. Events are ephemeral and trigger reactive behaviour.
State (static)	Represents a persistent condition or snapshot of data. State messages describe how things are, not what happened.

Data Structure

A Data Structure defines the schema of a Message payload. Data Structures are composed of typed fields.

3.3 Data Types

The following primitive and composite data types are supported for fields within Data Structures:

Category	Type	Notes
Scalar	bool	Boolean true / false
Scalar	string	UTF-8 character sequence
Scalar	char	Single character
Scalar	varchar	Variable-length string with defined max length
Numeric	int	Integer (platform-default width)
Numeric	float	Single-precision floating point
Numeric	double	Double-precision floating point
Temporal	date	Calendar date value
Composite	array	Ordered, indexed collection of a single type

Composite	tuple	Fixed-length, ordered, heterogeneous sequence
Composite	set	Unordered collection of unique values
Composite	map	Key-value associative structure
Composite	object	Named bag of typed fields
Reference	pointer	Reference to another data element
Custom	custom	User-defined type extending the base type system
Null	null	Explicit absence of a value

4 Solution Development Environment (SDE)

An SDE is the foundational deployable unit of Qala — a self-contained, configurable environment that defines all conditions for creating and operating a solution.

4.1 SDE Characteristics

Every SDE exhibits three core properties:

Property	Description
Deployable	An SDE can be deployed to any target: a platform, an environment (cloud, on-premise, hybrid), or a specific machine.
Configurable	All aspects of an SDE — tooling, languages, variables, preferences — are parameterised and adjustable without structural change.
Distributable	SDEs can be packaged, shared, replicated, and instantiated in new contexts, enabling consistent environments across teams.

4.2 SDE Scope

An SDE is solution-agnostic. It defines the conditions for solution work without embedding any solution-specific content. A single SDE can host and support a collection of solutions simultaneously.

> *The SDE is to Qala what a kernel is to an operating system: it provides the managed substrate on which solutions live, without being the solution itself.*

4.3 SDE Responsibilities

An SDE governs the following operational concerns:

Concern	Description
Creation	Bootstrapping and initialising new solutions within the environment.
Maintenance	Ongoing management of solution artefacts, dependencies, and configurations.
Lifecycle Management	Versioning, snapshotting, rollback, archiving, and decommissioning of solutions.
Operational Management	Runtime monitoring, health checks, alerting, and resource management.
Solution Replication	Cloning or templating environments for new teams, projects, or deployment targets.

4.4 SDE Composition

An SDE is composed of the following constituent elements:

Element	Description
Environment Variables	Runtime configuration values injected into the environment.
IDEs	Integrated development environments and editor configurations.
Languages	Supported programming languages, runtimes, and compilers.
User Preferences	Per-user settings, themes, and workflow preferences.
Configuration	System-level and service-level configuration files.
Profiles	Named configurations for different deployment targets or roles.
Settings	Platform-level operational settings.
Parameters	Tuneable values that modify SDE behaviour at runtime or boot.
Options	Feature flags and optional capability toggles.
Toolkits	Organised collections of Toolsets. Each Toolset contains individual Tools. Tools can be linked together to form Toolchains.
Solution Documentation	Charters, design documents, architectural records, and reference files.
Content Management	A solution-scoped CMS for managing all content artefacts produced within the SDE.

4.5 Toolkit Hierarchy

Tools within an SDE are organised into a three-level hierarchy:

- Toolkit: A named, purpose-oriented collection of related Toolsets (e.g., "Build Toolkit", "Security Toolkit").
- Toolset: A grouped set of Tools addressing a specific concern within the Toolkit.
- Tool: An individual executable capability (e.g., compiler, linter, scanner, formatter).

When Tools are linked together in sequence or in dependency graphs, they form Toolchains. Toolchains define automated, repeatable workflows such as build pipelines, test suites, or deployment sequences.

4.6 SDE Lifecycle

Every SDE passes through a defined set of lifecycle states:

State	Description

Provisioning	The SDE is being initialised: infrastructure allocated, base configuration applied.
Active	The SDE is running and available for solution work.
Snapshotted	A point-in-time capture of the SDE state has been taken. The SDE continues running.
Suspended	The SDE is paused. Resources are retained but not actively executing.
Rolled Back	The SDE has been restored to a prior snapshot state.
Archived	The SDE is frozen and stored for audit or future reference. No active work occurs.
Decommissioned	The SDE has been permanently shut down and resources released.

4.7 SDE Deployment Targets

An SDE can be deployed to any of the following target classes:

- Platform: A managed cloud or SaaS platform (e.g., AWS, GCP, Azure, Kubernetes cluster).
- Environment: A logical environment designation (e.g., development, staging, production).
- Machine: A specific physical or virtual machine (local workstation, on-premise server, edge node).

5 Solution Factory (SF)

A Solution Factory is a coordinated, networked collection of SDEs operating under shared governance to produce solutions in a consistent, repeatable, and scalable manner.

5.1 SF Characteristics

- An SF is formed by connecting two or more SDEs into a networked topology.
- SDEs within an SF are orchestrated: the Factory coordinates work distribution, resource allocation, and lifecycle events across all member SDEs.
- The SF provides a unified operational view across all constituent SDEs, enabling cross-environment analytics, security monitoring, and governance.
- A single SDE can belong to multiple Solution Factories.

5.2 SF Responsibilities

Responsibility	Description
SDE Orchestration	Coordinates the lifecycle, provisioning, and operational state of all member SDEs.
Consistency Enforcement	Ensures solutions produced across SDEs adhere to shared standards, toolchains, and governance policies.
Replication and Scaling	Enables solutions and SDEs to be replicated or scaled horizontally across the Factory.
Unified Observability	Aggregates metrics, events, and logs from all SDEs into a single operational view.
Cross-SDE Security	Applies security policy and threat monitoring uniformly across all SDEs in the Factory.
Artefact Coordination	Manages the sharing, promotion, and distribution of solution artefacts between SDEs.

6 System Architecture

Qala is implemented as a polyglot microservices platform organised into four planes, each implemented in the language best suited to its operational characteristics.

6.1 Architectural Planes

Plane	Language	Responsibilities
Kernel System	Rust	Service registry, subsystem health monitoring, command bus, event aggregation.
Control Plane	Go	API gateway, user identity and RBAC, SDE lifecycle management, workspace/CMS, artefact and package management.
Execution Plane	Scala	CI/CD workflow engine, data platform and analytics, notification dispatch.
Intelligence & Security Plane	Rust	AI recommendation and analysis agents, security event management (SEM), threat detection and response.

6.2 Kernel System

The Kernel System is the low-level coordination layer of the platform. It provides three core subsystems:

- Registry: Tracks all registered services, their metadata, and heartbeat/health status.
- CommandBus: Processes kernel-level commands including set_subsystem_status and set_service_status.
- EventAggregator: Counts, classifies, and stores recent events per topic, providing a real-time event summary to dependent services.

6.3 Control Plane

The Control Plane exposes the primary external API surface and governs all management operations:

- Gateway: Ingress point and routing layer. All external requests enter through the Gateway, which authenticates, authorises, and routes to downstream services.
- Identity: Manages user accounts, authentication, and role-based access control (RBAC) metadata.
- SDE Management: Implements the full SDE lifecycle (provisioning, snapshotting, rollback, decommissioning) and manages solution-to-SDE associations and SF coordination.
- Workspace / CMS: Manages file and content storage for workspaces. Supports versioned content management for solution documentation and artefacts.
- Artefact / Package: Manages the metadata, storage references, upload/download, and lifecycle of compiled packages and binary artefacts.

6.4 Execution Plane

The Execution Plane handles all asynchronous, compute-intensive, and long-running operations:

- Workflow / CI-CD: Triggers, executes, and tracks CI/CD pipelines. Provides status, logs, and result propagation.
- Data Platform: Executes analytical data pipelines, aggregates metrics and logs, manages master data, and serves analytics queries.
- Notifications: Dispatches notifications to users and systems via configurable channels (portal, email, Slack, Teams).

6.5 Intelligence & Security Plane

The Intelligence & Security Plane provides AI-powered insight and continuous security enforcement:

- AI Agents: Deliver SDE optimisation recommendations, predict bottlenecks, suggest solution structures, and detect configuration anomalies.
- SEM (Security Event Manager): Monitors for threats, enforces security policies, manages the secrets vault, initiates SDE quarantine when threats are confirmed, and provides audit-ready security event logs.

6.6 Repository Structure

Path	Contents
contracts/events/	Shared Kafka event schemas (JSON Schema).
contracts/openapi/	OpenAPI specifications for all REST endpoints.
db/postgres/	SQL schema definitions and migration scripts.
docs/	Architecture documentation and references.
go/	Control Plane microservices (Go).
rust/	Kernel System and Intelligence & Security Plane (Rust).
scala/	Execution Plane services (Scala).
infra/	Docker Compose, Kubernetes manifests, and Terraform configurations.

7 API Design

All external interaction with Qala is mediated through a unified REST API surface exposed by the Gateway service.

7.1 User Management

Endpoint	Description
POST /users	Create a new user account.
GET /users/{id}	Retrieve user profile and metadata.
PUT /users/{id}	Update user profile.
DELETE /users/{id}	Deactivate and remove a user account.

7.2 SDE Management

Endpoint	Description
POST /sdes	Provision a new SDE.
GET /sdes/{id}	Retrieve SDE configuration and status.
PATCH /sdes/{id}	Update SDE configuration.
DELETE /sdes/{id}	Decommission an SDE.
POST /sdes/{id}/snapshot	Take a point-in-time snapshot of an SDE.
POST /sdes/{id}/rollback	Restore an SDE to a prior snapshot.
POST /sdes/{id}/solutions	Associate a solution with an SDE.
DELETE /sdes/{id}/solutions/{sid}	Remove a solution association from an SDE.

7.3 Solution Management

Endpoint	Description
POST /solutions	Create a new solution record.
GET /solutions	List all solutions (with filtering).
GET /solutions/{id}	Retrieve a specific solution.
PATCH /solutions/{id}	Update solution metadata.
DELETE /solutions/{id}	Archive or delete a solution.

7.4 Solution Factory Management

Endpoint	Description
POST /factories	Create a new Solution Factory.
GET /factories	List all Solution Factories.
GET /factories/{id}	Retrieve a specific Factory.
PATCH /factories/{id}	Update Factory configuration.
DELETE /factories/{id}	Dissolve a Solution Factory.
POST /factories/{id}/sdes	Add an SDE to a Factory.
DELETE /factories/{id}/sdes/{sdeId}	Remove an SDE from a Factory.

7.5 Workspace & Content

Endpoint	Description
POST /workspaces	Create a new workspace.
GET /workspaces/{id}	Retrieve workspace metadata.
PUT /workspaces/{id}	Update workspace configuration.
DELETE /workspaces/{id}	Delete a workspace.
POST /workspaces/{id}/content	Upload or update content within a workspace.

7.6 Workflow & Artefacts

Endpoint	Description
POST /pipelines/run	Trigger a CI/CD pipeline.
GET /pipelines/{id}/status	Query pipeline execution status.
POST /artifacts/upload	Upload an artefact to the registry.
GET /artifacts/{id}	Retrieve artefact metadata.
GET /artifacts/{id}/download	Download an artefact binary.
DELETE /artifacts/{id}	Remove an artefact.

7.7 Intelligence & Security

Endpoint	Description
GET /recommendations	Retrieve AI-generated SDE/solution recommendations.
POST /analyze_sde	Request an AI analysis of a specific SDE.
GET /threats	List active and historical security threats.

POST /policy/update	Push a new security policy to the SEM.
POST /scan_sde	Initiate a security scan on an SDE.
POST /notify	Dispatch a notification.
GET /notifications/{id}	Retrieve a notification record.
POST /data/pipeline/run	Trigger a data analytics pipeline.
GET /data/analytics	Query analytics output.

8 Event Architecture

Qala is event-driven. All significant state changes and operational signals are published as events to a central message broker (Kafka), enabling loose coupling, auditability, and reactive processing.

8.1 Event Topics

Topic	Publisher / Consumer
USER_EVENTS	Identity service publishes; SDE, Workspace, and Notification services consume.
SDE_EVENTS	SDE Management service publishes; AI, SEM, Notification, and Kernel services consume.
CMS_EVENTS	Workspace/CMS service publishes; Notification and Analytics services consume.
BUILD_EVENTS	Workflow/CI-CD service publishes; Artefact, Notification, and AI services consume.
ARTIFACT_EVENTS	Artefact service publishes; Data Platform and Notification services consume.
DATA_EVENTS	Data Platform service publishes; Analytics and Notification services consume.
SECURITY_EVENTS	SEM service publishes; Notification, SDE Management, and Kernel services consume.
NOTIFICATIONS	All services can publish; Notification service routes and dispatches.
AI_RECOMMENDATIONS	AI Agents publish; SDE Management and user-facing services consume.

8.2 Event Schema

All events conform to the shared schema defined in contracts/events/schema.json. Every event envelope includes:

- event_id: Unique event identifier (UUID v4).
- topic: The Kafka topic name.
- event_type: A string classifying the event within its topic (e.g., SDE_CREATED, THREAT_DETECTED).
- timestamp: ISO 8601 UTC timestamp of event creation.
- source_service: The originating microservice.
- payload: Topic-specific structured data conforming to the topic schema.

9 Security Model

9.1 Authentication & Authorisation

- All API requests are authenticated via OAuth2 / JWT tokens issued by the Identity service.
- Multi-factor authentication (MFA) is supported and recommended for all human users.
- Role-Based Access Control (RBAC) governs all resource access. Roles are scoped at Factory, SDE, and Solution levels.
- Service-to-service communication uses mTLS within the platform perimeter.

9.2 Security Event Management (SEM)

The SEM subsystem provides continuous, active security across all SDEs and Factories:

- Threat Detection: Continuous analysis of SDE behaviour, network traffic, and configuration state for anomalies.
- Policy Enforcement: Security policies are defined centrally and pushed to all SDEs. Policy violations generate SECURITY_EVENTS.
- Secrets Vault: Credentials, API keys, and sensitive configuration values are stored in and injected from the Secrets Vault. No secrets are stored in plain text within SDE configuration.
- SDE Quarantine: Upon confirmed threat detection, the SEM can automatically isolate an SDE, preventing ingress/egress until the threat is resolved and the SDE passes a clean scan.
- Audit Trail: All security-relevant actions (policy updates, scans, quarantine events, access changes) are immutably logged.

9.3 Data Security

- All data is encrypted at rest using AES-256.
- All data in transit is encrypted using TLS 1.3 or higher.
- Database access is restricted to service accounts with least-privilege permissions.

10 AI & Intelligence

10.1 AI Agent Capabilities

The AI Agents subsystem provides the following intelligence hooks, available to all SDEs within a Factory:

Hook	Description
ai_optimize_sde_config	Analyses current SDE configuration and recommends optimisations for performance and resource efficiency.
ai_predict_bottleneck	Uses historical build and runtime data to predict upcoming performance or capacity bottlenecks.
ai_suggest_folder_structure	Recommends solution directory and project structure based on solution type and language profile.
anomaly_detection	Identifies anomalous patterns in SDE metrics, build outputs, or runtime behaviour.
resource_prediction	Forecasts future resource consumption based on solution growth trends.

10.2 Recommendation Lifecycle

Recommendations generated by the AI Agents are published to the AI_RECOMMENDATIONS Kafka topic and surfaced to users through the /recommendations API endpoint. Users can accept, dismiss, or defer recommendations. Accepted recommendations are applied directly to SDE configuration where automation is supported.

11 Observability

11.1 Logging

- All services emit structured JSON logs to a centralised log aggregation system.
- Log levels: DEBUG, INFO, WARN, ERROR, FATAL.
- All logs include: service name, trace ID, span ID, timestamp, and log level.

11.2 Metrics

- Service-level metrics (request rate, error rate, latency) are emitted to Prometheus.
- SDE-level metrics (resource utilisation, active solutions, build frequency) are computed by the Data Platform.
- All metrics are visualised in Grafana dashboards accessible through the Portal.

11.3 Distributed Tracing

- All inter-service calls are instrumented with distributed tracing (OpenTelemetry compatible).
- Trace context is propagated through all event payloads, enabling end-to-end request tracing from API entry to event emission.

12 Infrastructure & Deployment

12.1 Local Development

The full platform stack can be run locally using Docker Compose. The compose file at infra/docker-compose.yml brings up Kafka, PostgreSQL, and all microservices. Services are accessible on documented default ports (see docs/architecture.md).

Command	Action
docker-compose up -d	Start Kafka, PostgreSQL, and all services.
go run ./go/...	Run Control Plane Go services.
cargo run --manifest-path rust/Cargo.toml	Run Kernel and Intelligence/Security Rust services.
sbt run (scala/)	Run Execution Plane Scala services.

12.2 SDE Lifecycle Commands

The following SDE lifecycle operations are available via the API and CLI toolchain:

- snapshot_sde(sde_id): Capture a point-in-time snapshot of the named SDE.
- rollback_sde(sde_id, version): Restore the SDE to the specified snapshot version.
- save_template(sde_id): Persist the current SDE configuration as a reusable template.
- load_template(template_id): Instantiate a new SDE from a saved template.

12.3 Multi-Cloud Support

- SDEs can be deployed to any major cloud provider (AWS, GCP, Azure) or on-premise infrastructure.
- Infrastructure as Code definitions (Terraform) are provided for all major deployment targets.
- Kubernetes manifests are provided for containerised deployments at any scale.

13 Delivery Roadmap

Year 1

- Q1: Core API layer, PostgreSQL schemas, User Identity service.
- Q2: SDE Management (local), Workspace/CMS, basic CI/CD pipeline support.
- Q3: AI Phase 1 (recommendations), SEM Phase 1 (threat detection), IDE integrations.
- Q4: Artefact Management, developer onboarding documentation, portal dashboard.

Year 2

- Q1-Q2: Data pipelines, advanced analytics, CI/CD enhancements.
- Q3-Q4: AI Phase 2 (predictive analytics), SEM Phase 2 (automated quarantine), multi-cloud SDE deployment.

Year 3

- Q1-Q2: Full multi-cloud SDE orchestration, Solution Factory marketplace (templates, plugins, extensions).
- Q3-Q4: Global scaling, enterprise compliance add-ons, monetisation and SaaS packaging.

Key Performance Indicators

KPI	Target
Developer Onboarding Time	New SDE provisioned and first solution created within 30 minutes.
CI/CD Pipeline Success Rate	> 95% pipeline success rate across all active SDEs.
SDE Snapshot / Rollback Latency	Snapshot completed within 60 seconds; rollback within 5 minutes.
AI Recommendation Acceptance Rate	> 40% of AI recommendations accepted within 7 days of generation.
Threat Detection to Quarantine Latency	< 90 seconds from threat signal to SDE quarantine initiation.
Multi-Cloud SDE Provisioning Time	< 10 minutes for any supported cloud target.

14 Glossary

Term	Definition
SDE	Solution Development Environment. A deployable, configurable, distributable environment defining all conditions for solution creation and management.
SF	Solution Factory. A networked, orchestrated collection of SDEs producing solutions in a consistent and scalable manner.
Solution	Any system, application, good, product, service, or platform produced within the Qala platform.
Toolchain	A linked sequence of Tools within an SDE, forming an automated workflow.
Toolkit	A named collection of Toolsets within an SDE.
Toolset	A grouped set of Tools addressing a specific concern within a Toolkit.
Interface	The contract surface of a Component, declaring its imports and exports.
Message	The unit of communication flowing through an Interface, typed as either Event (dynamic) or State (static).
SEM	Security Event Manager. The Qala subsystem responsible for threat detection, policy enforcement, and SDE security management.
RBAC	Role-Based Access Control. The authorisation model governing all resource access within Qala.
Artefact	A compiled, packaged, or binary output produced by a solution build process.
Snapshot	A point-in-time capture of an SDE state, enabling rollback and auditing.
Quarantine	An isolated SDE state imposed by the SEM when an active threat is confirmed.
CMS	Content Management System. The content storage and versioning layer within each SDE.

End of Document — Qala Solution Factory OS v1.0